

Embedded Systems

GNC Software Onboarding Lecture #2

What is an Embedded System?

- ▶ An **embedded system** is a computer system that differs from a desktop computer, laptop, or similar in that it has limited resources
 - ▶ Often limited in size, computational power, memory
- ▶ Typically use the ARM processor architecture instead of Intel/x86 processors found on modern desktop/laptop computers
- ▶ Instead of central processing units (CPUs), we call their embedded counterparts microcontrollers (**MCUs**) and microprocessors (**MPUs**)
 - ▶ MPUs are similar to MCUs, except MPUs typically have more processing power and memory available to them



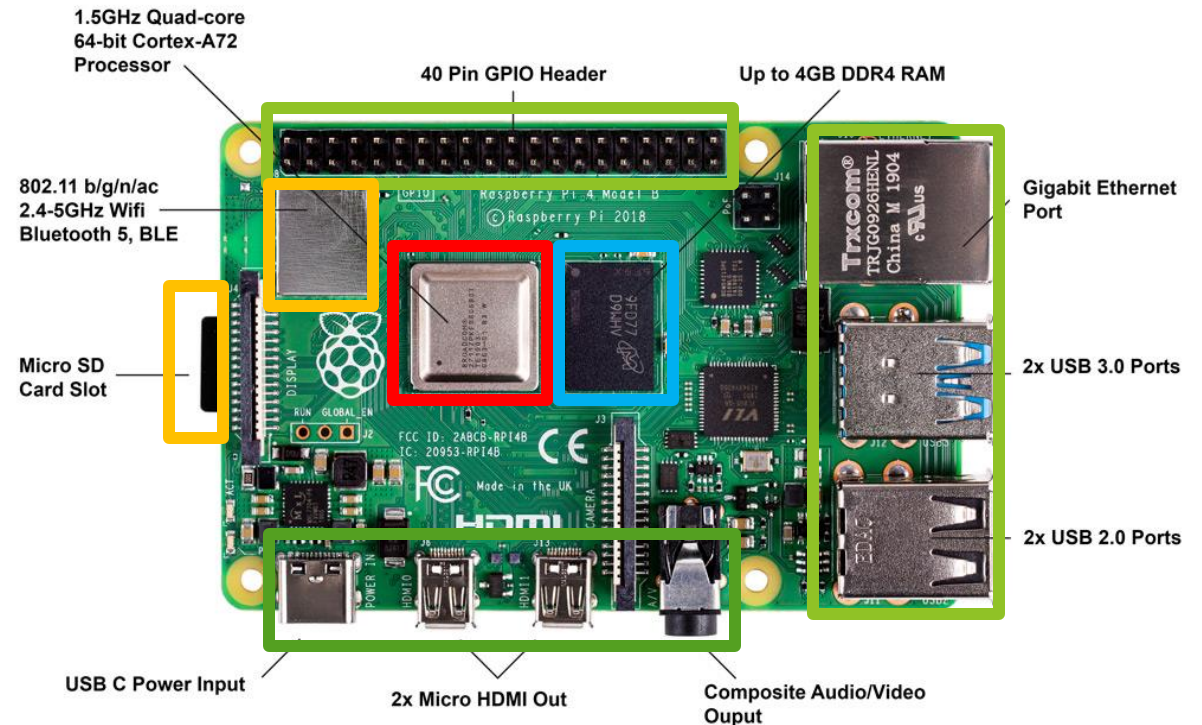
Example of an MCU



Bottom of a CPU

Components of an Embedded System

- ▶ **MCU/MPU** - Typically identifiable as the biggest chip with the most pins on the board
- ▶ **GPIOs/ports** - metal contacts exposed through headers, PCB traces, or I/O ports like USB or Ethernet
 - ▶ All these connections lead directly back to the pins on the MCU/MPU
- ▶ **Memory** - sometimes integrated on the MCU/MPU itself, sometimes connected separately through additional GPIOs
- ▶ **Peripherals** - sensors, actuators, displays, human interface devices (keyboards, mice, etc), buttons, speakers, microphones, etc.



Raspberry Pi

How does the MCU interact with the outside world?

- ▶ The MCU itself is just a black box that can generate high and low digital signals on each one of its pins
 - ▶ How can we leverage this to cause outside side effects?
- ▶ Basic GPIO functions
 - ▶ Turning an LED on by writing a GPIO pin high
 - ▶ Igniting a rocket motor by writing a GPIO pin high
 - ▶ Responding to a button push through a GPIO input
- ▶ Communication with peripherals
 - ▶ Typically multiple GPIO pins dedicated to sending data to and from a specific peripheral
 - ▶ We have standard ways of doing this called communication protocols

A Simple Communication Protocol

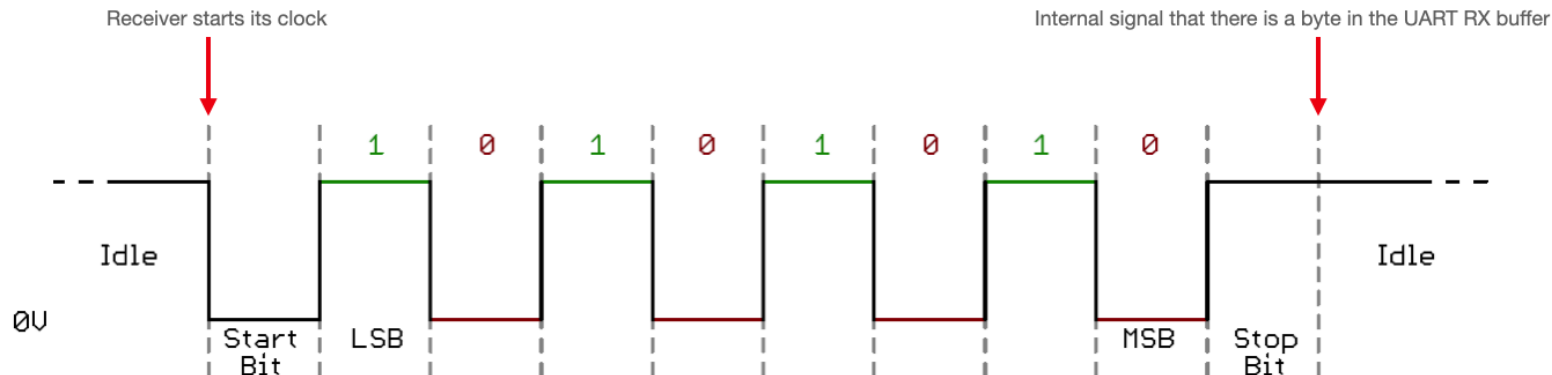
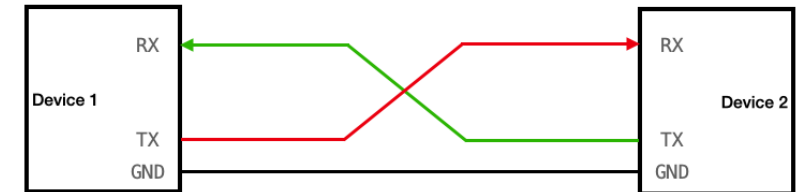
- ▶ Say device A wants to send a message to device B
- ▶ Let's dedicate one wire as an output on A and an input on B
- ▶ At the lowest level, any form of communication is just a structured stream of information transmitted through a sequence of bits
 - ▶ Ex. The message 'Hi' could be encoded using the the sequence '0100100001101001'
- ▶ Define a zero as when the wire has a low voltage, and a one as when the wire has a high voltage
- ▶ Every millisecond, transmit the next bit by changing the voltage on the wire to represent the next bit in the message
 - ▶ Our sequence would be low, high, low, low, high, low, low, low, ...

A few problems with our solution...

- ▶ What if we don't want to send any message at all?
 - ▶ Currently, holding the wire at a low or high voltage would transmit a steady sequence of 0's or 1's
- ▶ What if we want to transmit data at a faster/slower speed?
- ▶ How can we send data from device B to device A?

UART

- ▶ The Universal Asynchronous Receiver/Transmitter (**UART**) protocol is one of the simplest embedded protocols
- ▶ Two signal wires: RX and TX
- ▶ RX and TX operate identically, but the only difference is the direction of the message being sent
- ▶ Wire is high when idle, and a device will pull its TX pin low to 0 when it wants to send a byte
- ▶ At the end of a byte, set the pin back high
- ▶ Bits are then sent at an agreed upon rate called the baud rate



How does UART solve these problems?

- ▶ What if we don't want to send any message at all?
 - ▶ We can hold the UART line high to indicate that it is idle
 - ▶ When we send a zero to indicate a message start, we send 8 bits and then the message is over
- ▶ What if we want to transmit data at a faster/slower speed?
 - ▶ Both devices agree on a baud rate
- ▶ How can we send data from device B to device A?
 - ▶ Separate RX and TX wires

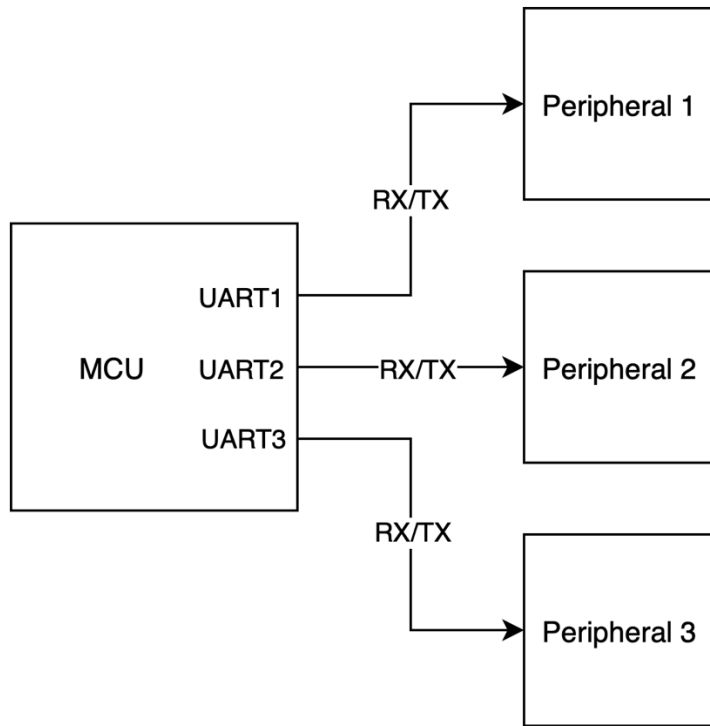
Communicating with multiple devices

- ▶ One problem with UART is that each device needs its own RX/TX pair with the MCU
- ▶ How can we use one wire that multiple devices share?
 - ▶ We need some way to coordinate access to this wire to not create a short circuit
 - ▶ When a device isn't actively communicating, it needs to not be driving a 1 or a zero on the shared wire
- ▶ We call this “shared wire” concept a bus

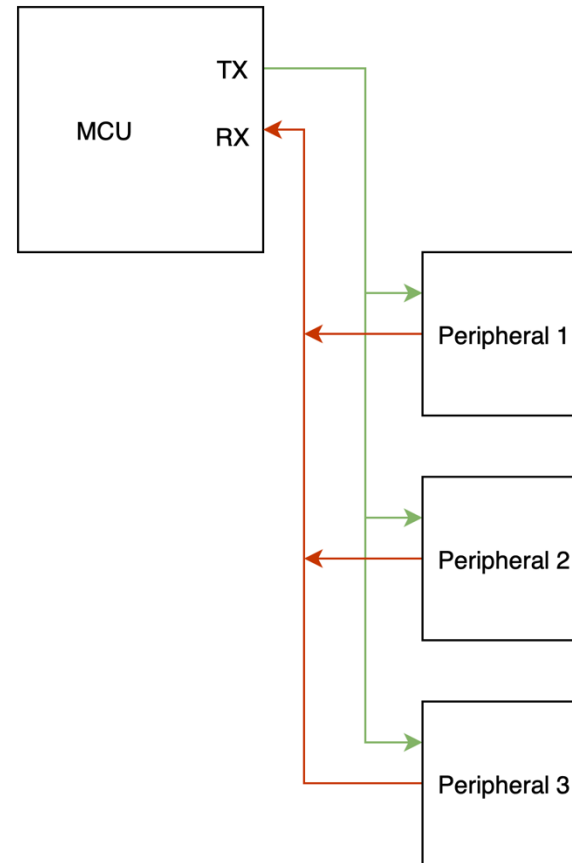
Buses

- ▶ When we said earlier that digital logic only has two states, that was an oversimplification
- ▶ In digital logic, there is a third state called floating/high impedance, often denoted by the letter 'Z'
 - ▶ Happens when there is nothing actively driving the wire high or low
- ▶ A bus can utilize this third logic state to have more than one device communicating on a single wire
- ▶ Still, only one device can drive the bus at a time
 - ▶ All other devices must have their output as Z while a device is outputting on the bus

Buses (Continued)



RX/TX pair per peripheral



RX/TX pair on a bus

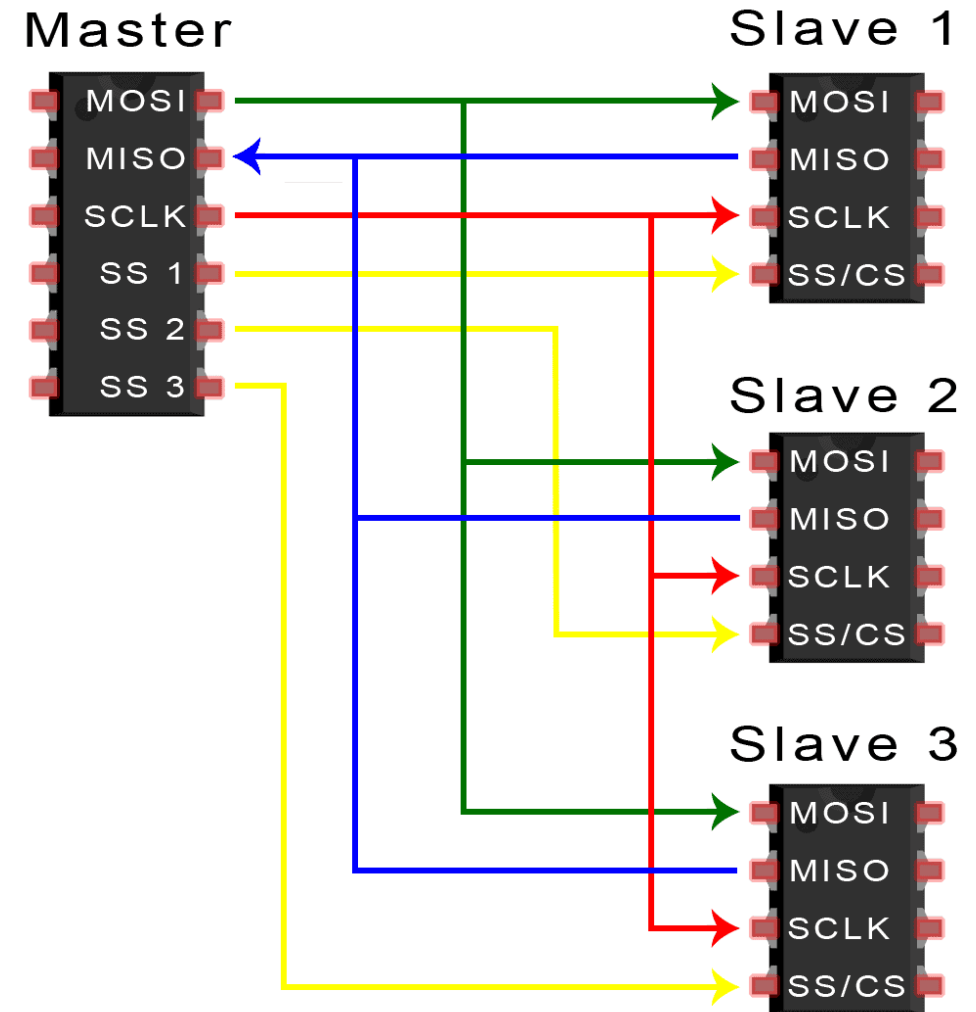
*note that UART typically cannot function on a bus. This is just an example

SPI

- ▶ The Serial Peripheral Interface (**SPI**) introduces the concept of a master and a slave
 - ▶ Master is the MCU, slave is the peripheral
- ▶ Four wires
 - ▶ Master Out Slave In (MOSI) carries data from the master to the slave (BUS)
 - ▶ Master In Slave Out (MISO) carries data from the slave to the master (BUS)
 - ▶ Serial Clock (SCLK) controls the rate at which data is sent
 - ▶ Slave Select / Chip Select (SS/CS) - low when the master activates the slave, high when inactive
- ▶ There is no concept of start/stop bytes as there is in UART
 - ▶ Data is streamed continuously between master and slave
- ▶ On the same SPI bus, multiple slaves can be connected, but they all need their own SS/CS line
 - ▶ So, an SPI bus would consist of shared MOSI, MISO, and SCLK lines

SPI (Continued)

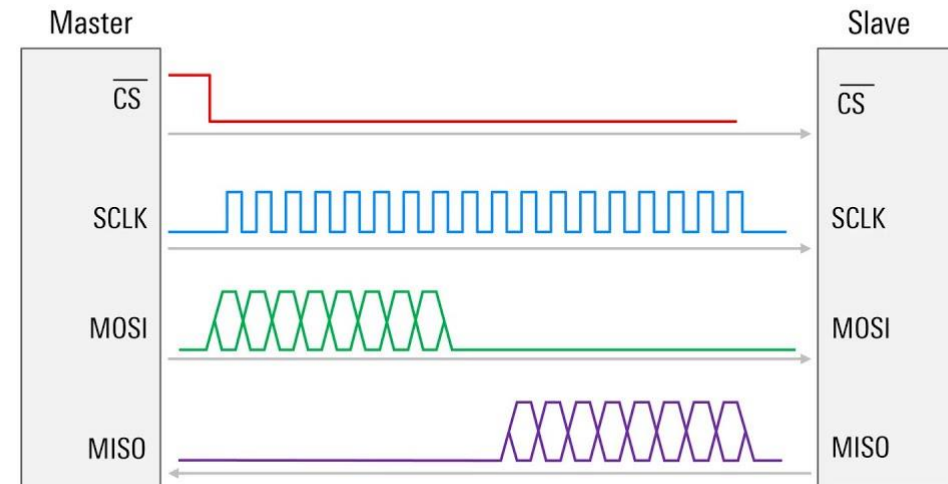
- Master Out Slave In (MOSI) carries data from the master to the slave (BUS)
- Master In Slave Out (MISO) carries data from the slave to the master (BUS)
- Serial Clock (SCLK) controls the rate at which data is sent
- Slave Select / Chip Select (SS/CS) - low when the master activates the slave, high when inactive



SPI (Continued)

- ▶ Master Out Slave In (MOSI) carries data from the master to the slave (BUS)
- ▶ Master In Slave Out (MISO) carries data from the slave to the master (BUS)
- ▶ Serial Clock (SCLK) controls the rate at which data is sent
- ▶ Slave Select / Chip Select (SS/CS) - low when the master activates the slave, high when inactive

Overview of SPI protocol



I2C

- ▶ Inter-Integrated Circuit (I2C) protocol
- ▶ Two wires: SDA and SCL
 - ▶ SDA (Serial Data) carries the data (BUS)
 - ▶ SCL (Serial Clock) carries a clock signal
- ▶ As in SPI, multiple slaves can be connected to the same SDA and SCL lines
 - ▶ We distinguish between slaves by giving them unique 8 byte addresses
 - ▶ A device will only take control of the bus if it detects its address on the bus

Responding to Real-Time Events

- ▶ Imagine a system where a button press needs to cause some immediate action to be taken
- ▶ We could read the GPIO input every so often and perform the action when we read it as high (polling)
 - ▶ Waste of computation time
 - ▶ Now our entire software is bound to this checking loop, since we must always be ready to respond to the event
- ▶ **Interrupts** eliminate the need to poll by stopping the regular flow of code execution at a hardware level
- ▶ Once the button is pressed, an interrupt will be triggered and the MCU will jump to a special function called an Interrupt Service Routine (**ISR**)
 - ▶ We then perform our action in the ISR
- ▶ Code that was being executed when the interrupt fires resumes execution as if nothing ever happened

Interrupts and Communication Protocols

- ▶ Say our MCU is waiting for data from another device over UART. How can we tell when data is available for us to process?
- ▶ Create an ISR that fires when we receive a byte over UART
 - ▶ From that ISR, perform whatever processing we need to do on that byte

STM32 HAL

- ▶ In GNC, we typically use MCUs from a company called STMicroelectronics (STM)
 - ▶ Their MCUs are broadly named “STM32”
- ▶ While the physical details of the chips differ across specific STM32 products, they are similar enough that STM offers a generic library for their line of MCUs called the Hardware Abstraction Library (**HAL**)
- ▶ It contains functions to control all of the communication protocols we covered previously, and then some
- ▶ Allows for control of individual GPIOs, writing them high or low
- ▶ Also includes many other on-chip peripherals not covered in this presentation

Practice Exercises

- ▶ Go to <https://github.gatech.edu/gnc/software-onboarding> and do exercise 2
 - ▶ 1. Run “git clone <https://github.gatech.edu/gnc/software-onboarding>”
 - ▶ 2. Create a branch by running “git checkout -b <first name>-exercise2”
 - ▶ Ex. Sam would run “git checkout -b sam-exercise2”
 - ▶ This creates a new branch
 - ▶ 3. Do the exercise following the instructions in the exercise 2 README
 - ▶ 4. Stage your changes by running “git add .”
 - ▶ 5. Commit your changes by running “git commit -m <commit message>”
 - ▶ Ex. git commit -m “Sam exercise 2”
 - ▶ 6. Push your changes by running “git push origin <first name>-exercise2”
 - ▶ 7. Create a pull request in GitHub to merge your branch into the main branch

Reference Materials

Useful Links

- ▶ https://www.st.com/resource/en/user_manual/um2217-description-of-stm32h7-hal-and-lowlayer-drivers-stmicroelectronics.pdf - STM32 HAL Reference for H7 MCUs

I2C (Continued)

- Steps in I2C communication
 - 1. Start Bit - Master pulls the idle SDA line low before SCL goes low
 - 2. Slave Address - Master transmits the address of the slave to communicate with
 - 3. Read/Write - Master sends a 1 to read data, 0 to write data
 - 3. Slave Acknowledge - Slave sends a 0 to accept, 1 to reject the request
 - 4. Data - Master or slave 8 bits, depending on read/write
 - 5a (Read). Master Acknowledge - Master sends a 0 to accept, 1 to reject the data
 - 5b (Write). Slave Acknowledge - Slave sends a 0 to accept, 1 to reject the data
 - 6. Stop bit - Master releases SDA line to high after SCL goes high

Overview of I2C frames

